

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 06 -

DEPENDENCY INVERSION PRINCIPLE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS	12

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o Princípio da Inversão de Dependência (Dependency Inversion Principle), que é o quinto dos cinco princípios SOLID da programação orientada a objetos e design de software. Vamos entender a importância de depender de abstrações em vez de implementações concretas para criar um código flexível e desacoplado.



HANDS ON

Nesta aula prática, compreenderemos o DIP, aprendendo o conceito de inversão de dependência e por que ele é fundamental para a criação de um código modular e flexível.

Identificaremos violações do DIP, reconhecendo situações em que o código depende diretamente de implementações concretas, aumentando o acoplamento e dificultando a manutenção. Depois, vamos refatorar um código aplicando o DIP. Veremos exemplos práticos de como refatorar o código para que ele siga o Princípio da Inversão de Dependência, utilizando interfaces e injeção de dependência.

SAIBA MAIS

O Princípio da Inversão de Dependência (DIP) afirma que os módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações. Além disso, as abstrações não devem depender de detalhes. Os detalhes devem depender de abstrações.

A ideia principal do DIP é evitar a dependência direta de classes de alto nível em classes de baixo nível. Em vez disso, ambas as classes devem depender de interfaces ou abstrações, promovendo um design mais flexível e desacoplado.

Esse é um dos princípios que toda pessoa desenvolvedora ou arquiteta deveria conhecer, pois ele é a base de arquiteturas famosas, como a Arquitetura Limpa, Arquitetura Hexagonal, Onion Architecture e Vertical Slice. Nós focamos nesse princípio para diminuir o acoplamento entre as camadas.

Assim como nas aulas anteriores, vamos analisar um exemplo que viola o DIP e depois refatorá-lo, para que possamos entender esse princípio na prática.

Iniciando pela violação, aqui temos um exemplo de uma classe `NotificationService` que não segue o Princípio da Inversão de Dependência, pois depende diretamente de uma implementação concreta `EmailService`. Essa abordagem torna o código menos flexível e mais difícil de manter.

Classe `EmailService`:

```
public class EmailService {  
    public void sendEmail(String message) {  
        System.out.println("Email enviado: " + message);  
    }  
}
```

Código 1 — Exemplo
Fonte: elaborado pelo autor (2024)

`NotificationService`:

```
public class NotificationService {  
    private EmailService emailService;  
  
    public NotificationService() {  
        this.emailService = new EmailService();  
    }  
  
    public void sendNotification(String message) {  
        emailService.sendEmail(message);  
    }  
}
```

Código 2 — Exemplo (2)
Fonte: elaborado pelo autor (2024)

Agora, para que possamos testar a nossa classe Main.java:

```
public class Main {  
    public static void main(String[] args) {  
        NotificationService notificationService = new NotificationService();  
        notificationService.sendNotification("Olá, aluno FIAP!");  
    }  
}
```

Código 3 — Exemplo (3)
Fonte: elaborado pelo autor (2024)

Testando esse código, temos o seguinte resultado:

```
(base) thiagoadriano@Thiagos-MBP aula5 %  
(base) thiagoadriano@Thiagos-MBP aula5 % cd /Users/thiagoadriano/Documents/FIAP/  
/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/thiag  
acdf1b981/redhat.java/jdt_ws/aula5_73382870/bin Main  
Email enviado: Olá, aluno FIAP!  
(base) thiagoadriano@Thiagos-MBP aula5 %
```

Figura 1 — Resultado de execução código violando o DIP
Fonte: elaborado pelo autor (2024)

A proposta inicial não era verificar se o código funciona ou não, mas vamos fazer uma análise rápida. Você pode fazer uma comparação com um código que você tenha em produção e esteja funcionando, mas infringindo esse princípio. Você pode pensar que “Em time que está ganhando não se mexe”, correto?

Vamos refatorar este exemplo, para que possamos mudar esse pensamento juntos. Vamos criar uma nova interface chamada `MessageService`:

```
package src;

public interface MessageService {
    void sendMessage(String message);
}
```

Código 4 — Nova interface
Fonte: elaborado pelo autor (2024)

Agora, vamos implementá-la em `EmailService`:

```
package source;

public class EmailService implements MessageService {
    @Override
    public void sendMessage(String message) {
        System.out.println("Email enviado: " + message);
    }
}
```

Código 5 — Implementação em `EmailService`
Fonte: elaborado pelo autor (2024)

Atualizando a nossa classe `NotificationService`:

```
public class NotificationService {
    private EmailService emailService;

    public NotificationService() {
```

```
this.emailService = new EmailService();  
}  
  
public void sendNotification(String message) {  
    emailService.sendEmail(message);  
}  
}
```

Código 6 — Atualizando classe
Fonte: elaborado pelo autor (2024)

Para que possamos testar, vamos atualizar a nossa classe Main:

```
public class Main {  
    public static void main(String[] args) {  
        NotificationService notificationService = new NotificationService();  
        notificationService.sendNotification("Olá, aluno FIAP!");  
    }  
}
```

Código 7 — Atualizando classe Main
Fonte: elaborado pelo autor (2024)

Resultado da execução do código:

```
(base) thiagoadriano@Thiagos-MBP aula5 %  
● (base) thiagoadriano@Thiagos-MBP aula5 % cd /Users/thiagoadriano/  
/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages  
acdf1b981/redhat.java/jdt_ws/aula5_73382870/bin Main  
Email enviado: Olá, aluno FIAP!  
○ (base) thiagoadriano@Thiagos-MBP aula5 %
```

Figura 2 — Executando código que refatoramos utilizando o DIP
Fonte: elaborado pelo autor (2024)

Depois dessa refatoração, ficou mais fácil adicionar uma nova implementação.

Para ficar claro, vamos criar uma nova chamada **SMSService** para enviar SMS, mostrando como é fácil adicionar novas funcionalidades sem modificar o código existente.

Classe SMSService:

```
package src;

public class SMSService implements MessageService {
    @Override
    public void sendMessage(String message) {
        System.out.println("SMS enviado: " + message);
    }
}
```

Código 8 — Classe SMSService
Fonte: elaborado pelo autor (2024)

Testando o fluxo adicionando-o na classe Main:

```
package source;

public class Main {
    public static void main(String[] args) {
        // Usando EmailService
        MessageService emailService = new EmailService();
        NotificationService notificationServiceEmail = new
NotificationService(emailService);
        notificationServiceEmail.sendNotification("Olá, mundo via Email!");

        // Usando SMSService
        MessageService smsService = new SMSService();
        NotificationService notificationServiceSMS = new
NotificationService(smsService);
        notificationServiceSMS.sendNotification("Olá, mundo via SMS!");
    }
}
```

Código 9 — Testando fluxo
Fonte: elaborado pelo autor (2024)

Executando o código:

```
(base) thiagoadriano@Thiagos-MBP aula5 %  
● (base) thiagoadriano@Thiagos-MBP aula5 % cd /Users/thiagoadriano/D  
/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp  
acdf1b981/redhat.java/jdt_ws/aula5_73382870/bin source.Main  
Email enviado: Olá, mundo via Email!  
SMS enviado: Olá, mundo via SMS!  
○ (base) thiagoadriano@Thiagos-MBP aula5 %
```

Figura 3 — Exemplo adicionando a classe SMSService
Fonte: elaborado pelo autor (2024)

Vamos a um resumo de todo esse fluxo. Antes da refatoração, a classe **NotificationService** estava violando o DIP ao depender diretamente de **EmailService**, uma implementação concreta.

Depois da refatoração:

1. **Interface **MessageService****: define um contrato para enviar mensagens, permitindo diferentes implementações (por exemplo, email ou SMS, como vimos no passo anterior).
2. **Classe **EmailService****: implementa a interface **MessageService** para enviar emails.
3. **Classe **NotificationService****: depende da abstração **MessageService**, permitindo a injeção de qualquer implementação concreta de **MessageService**.

Benefícios da refatoração

- **Modularidade**: o código fica mais modular e fácil de modificar, pois podemos trocar a implementação de **MessageService** sem alterar **NotificationService**.
- **Flexibilidade**: facilita a adição de novas formas de envio de mensagens (como SMS ou notificações push) sem modificar o código existente.
- **Redução de acoplamento**: reduz o acoplamento entre classes de alto nível e classes de baixo nível, promovendo um design mais limpo e sustentável.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o Princípio da Inversão de Dependência (Dependency Inversion Principle), o quinto dos cinco princípios SOLID da programação orientada a objetos e design de software.



REFERÊNCIAS

MEDIUM. **Os Princípios do SOLID — DIP Princípio de inversão de dependência.** 2019. Disponível em: <https://medium.com/@jonesroberto/os-princ%C3%ADpios-do-solid-dip-princ%C3%ADpio-de-invers%C3%A3o-de-depend%C3%Aancia-7e110cfcc3e5>. Acesso em: 07 ago. 2024.

MEDIUM. **Solid — D.I.P — Dependency Inversion Principle.** 2019. Disponível em: <https://medium.com/@tbaragao/solid-d-i-p-dependency-inversion-principle-e87527f8d0be>. Acesso em: 07 ago. 2024.

NDEPEND. **SOLID Design in C#: The Dependency Inversion Principle (DIP) with Examples.** 2020. Disponível em: <https://blog.ndepend.com/solid-design-the-dependency-inversion-principle-dip/>. Acesso em: 07 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Dependency Inversion Principle, DIP, SOLID, Design de Software, Programação Orientada a Objetos, Abstrações, Injeção de Dependência, Modularidade, Flexibilidade, Redução de Acoplamento.

EXEMPLO



POSTECH